

客户端连接指南 — 使用 X.509 证书连接 UA Mock Server

本文写给客户端工程师：如何用证书连接由 ua_hda 提供的 OPC UA 模拟服务器。

该服务器强制 **X.509 客户端证书认证**：安全策略为 `Basic256Sha256_SignAndEncrypt`，只接受受信证书签发的客户端。**不带证书、或用未受信证书连接，都会被服务器拒绝**（`BadCertificateUntrusted`）。

1. 你会收到 3 个文件

文件	作用	建议放置
<code>client_cert.pem</code>	客户端身份证书（登录凭证）	证书库，或随客户端分发
<code>client_key.pem</code>	客户端私钥（与上证书配对）	随客户端分发， 勿入版本库、勿外传
<code>server_cert.pem</code>	服务端证书	客户端受信列表，用于校验服务器身份

2. 连接时要填的参数

- 端点地址：`opc.tcp://<服务器地址>:<端口>/ua_mockers/`（地址与端口由部署方另行提供）
- 安全策略：`Basic256Sha256_SignAndEncrypt`
- 证书加载：客户端证书 + 客户端私钥 + 服务端证书，三者同时配置

3. 连接成功意味着什么

- 服务器验过你的客户端证书（在受信链上）；
- 你也验过服务器证书（防止连错服务器 / 中间人）。

4. Python 示例（asyncua, 已实测通过）

以下为本仓库 `client_demo.py` 的真实可用代码：

```
import asyncio

from asyncua import Client, ua
from asyncua.crypto.security_policies import SecurityPolicyBasic256Sha256

URL = "opc.tcp://<服务器地址>:<端口>/ua_mockers/" # 改成部署方给的地址端口
```

```

SERVER_CERT = "certs/server_cert.pem"    # 收到的服务端证书
CLIENT_CERT = "certs/client_cert.pem"    # 收到的客户端证书
CLIENT_KEY = "certs/client_key.pem"      # 收到的客户端私钥

async def main():
    client = Client(URL)
    await client.set_security(
        SecurityPolicyBasic256Sha256,
        CLIENT_CERT,
        CLIENT_KEY,
        server_certificate=SERVER_CERT,
        mode=ua.MessageSecurityMode.SignAndEncrypt,
    )
    await client.connect()
    print("连接成功:", URL)

    # 读取示例节点（ns=1 命名空间下的变量）
    node = client.get_node("ns=1;s=int32_ch_1")
    print("int32_ch_1 =", await node.read_value())

    await client.disconnect()

if __name__ == "__main__":
    asyncio.run(main())

```

要点:

- `set_security()` 必须在 `connect()` 之前调用;
- `server_certificate` 填收到的 `server_cert.pem` , 客户端据此校验服务器;
- `mode` 用 `SignAndEncrypt` (与服务器策略一致) ;
- 连接用 `127.0.0.1` 以外的地址时, 证书不受影响, 照常可用。

5. 其他语言客户端的等价配置

证书使用逻辑与语言无关, 各客户端只需做同一件事:

1. 把 `client_cert.pem` + `client_key.pem` 注册为该客户端的 **应用证书 (Application Certificate)** ;
2. 把 `server_cert.pem` 加入客户端的 **受信服务器证书列表 (Trusted Server Certificates)** ;
3. 端点安全策略选 `SecurityPolicyBasic256Sha256` + `MessageSecurityMode.SignAndEncrypt` 。

例如 .NET (OPC UA .NET Standard)、C++ (open62541)、Java (Milo) 均在证书存储 / 应用配置里完成上述三步，证书文件 (PEM) 可直接导入，无需转格式。